

8-1-2010

The Perceived Advantage of Agile Development Methodologies By Software Professionals: Testing an Innovation-Theoretic Model

Nancy A. Bonner

University of Mary Hardin-Baylor, nbonner@umhb.edu

James T. C. Teng

University of Texas at Arlington, jtteng@uta.edu

Sridhar Nerur

University of Texas at Arlington, snerur@uta.edu

Follow this and additional works at: <http://aisel.aisnet.org/amcis2010>

Recommended Citation

Bonner, Nancy A.; Teng, James T. C.; and Nerur, Sridhar, "The Perceived Advantage of Agile Development Methodologies By Software Professionals: Testing an Innovation-Theoretic Model" (2010). *AMCIS 2010 Proceedings*. Paper 93.
<http://aisel.aisnet.org/amcis2010/93>

This material is brought to you by the Americas Conference on Information Systems (AMCIS) at AIS Electronic Library (AISeL). It has been accepted for inclusion in AMCIS 2010 Proceedings by an authorized administrator of AIS Electronic Library (AISeL). For more information, please contact elibrary@aisnet.org.

The Perceived Advantage of Agile Development Methodologies By Software Professionals: Testing an Innovation-Theoretic Model

Nancy A. Bonner

University of Mary Hardin-Baylor
nbonner@umhb.edu

James T. C. Teng

University of Texas at Arlington
jtteng@uta.edu

Sridhar Nerur

University of Texas at Arlington
snerur@uta.edu

ABSTRACT

Proponents of agile processes claim that agile practices result in higher quality software while allowing the flexibility to respond to evolving user requirements. Yet, to the best of our knowledge, no empirical study has really confirmed that benefits accrue to those who use agile processes. Grounded in the agility and diffusion of innovations literature, this research introduces a measure of process agility within the software development domain and relates it to constructs previously employed in the innovation literature. For agile methods, our study has provided empirical support for the proposal that agile development methods lead to developers' beliefs that they are less complex, more compatible and provide increased benefits. Since developers believe that agility leads to increased benefits, they will be more likely to accept agile methods. For practitioners, this study provides valuable insights into the underlying factors that influence a developer's beliefs about agile methods.

Keywords

Agility, Software Development, Methodology, Technology Acceptance, Process Innovation

INTRODUCTION

As organizations pour increasing amounts of resources into software development projects that, more often than not, come in over budget, overdue and deficient in terms of both quality and functionality (Benko and McFarlan, 2003; McDonald, 2001), it becomes increasingly important to investigate ways to improve software project success. One approach is to improve the software development process. A variety of new and innovative methods and tools designed to improve the process have been introduced and research studies examining these new methods have shown promising results in terms of increased quality, satisfaction and productivity (Nosek, 1998; Kessler, 1999; Parsons, et al. 2007; Williams, 2000; Williams, et al. 2000; Upchurch 2001). In the past, organizations and their developers have been reluctant to commit to their use (Glass 1999; Hardgrave 1995; Hartwick and Barki 1994). Understanding factors related to developers' acceptance of agile methods becomes increasingly important as agile methodologies continue to be adopted by organizations. In fact, the rapid ascent of agile methods (Ambler 2007) over the past few years indicates that a lot of organizations are in the process of adopting (or have recently adopted) these methodologies. In a recent global CIO study conducted by IBM, lean processes were cited as a key. The Agile Manifesto (Agile Alliance, 2001) articulates the fundamental doctrine of agile development methods. These basic tenets emphasize the value of individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation and responding to change over following a plan (Agile Alliance, 2001). Proponents of agile processes claim that agile practices result in higher quality software while allowing the flexibility to respond to evolving user requirements. Yet, to date, to the best of our knowledge, no empirical study has really confirmed that benefits accrue to those who use agile processes. Our study hopes to fill that void. The primary objectives of this research are to examine how complexity and compatibility mediate the relationship between process agility and its perceived benefits as well as to examine the direct effect of process agility on perceived benefits.

The paper is organized as follows. First, we provide a brief description of the background literature on process agility and diffusion of innovations. This is followed by the research model and hypotheses. Subsequent sections deal with measurements, the research methodology and the results. After a brief outline of the data analysis, we discuss our findings and the implications. Finally, we present the limitations of the study.

BACKGROUND LITERATURE

Process Agility

The concept of agility within the context of business was first introduced in a study conducted by the Iaccoca Institute. Within manufacturing, agility includes being able to bring new products to market and respond to customer needs rapidly by incorporating “innovative management structures, flexible technology and a skill base of knowledgeable workers” (Nagel and Dove, 1991). While agility originated within manufacturing research, its scope has widened to encompass other aspects of an organization such as agile supply chains (Swafford, 2003) and agile workforces (Breu *et al.*, 2001).

As mentioned earlier, the Agile Manifesto describes process agility within the context of agile software development. There are 12 principles behind the Agile Manifesto and these principles describe practices and beliefs that embody these four tenets.

Agile methodologies are designed to assist the developer in responding quickly and effectively to changes. The agile process is dynamic and strives to deliver working software in short iterations by anticipating and embracing change, actively involving stakeholders and continually seeking users’ feedback to address their evolving needs. Agile practices are characterized by collaborative teams that are able to adapt and respond to changing conditions through reaction capabilities such as flexibility of team members’ roles and the willingness to deviate from established processes in order to deliver working software. Agile capabilities involve the willingness to improvise and come up with new solutions on the fly.

Diffusion of Innovations

In his often cited book, *Diffusion of Innovations*, Rogers defines an innovation as “an idea, practice, or object that is perceived as new by an individual or other unit of adoption.” He goes further to say that it does not matter if the idea is “objectively” new as perceived by the lapse of time since its original development; if the idea, practice or object seems new to the individual, it is considered an innovation. Rogers defined 5 categories of variables believed to determine rate of adoption. They are perceived attributes of innovations, type of innovation-decision, communication channels, nature of the social system and extent of change agent’s promotion efforts. Of these 5 types, the perceived attributes of innovations have “been most extensively investigated” and “they have been found to explain about half of the variance” in rates of adoption (Rogers 2003). These 5 “perceived attributes of innovations” are:

- 1) relative advantage – the degree to which an innovation is perceived as better than the idea it supersedes,
- 2) compatibility – the degree to which an innovation is perceived as being consistent with the existing values, past experiences, and needs of potential adopters,
- 3) complexity – the degree to which an innovation is perceived as difficult to understand and use,
- 4) trialability – the degree to which an innovation may be experimented with on a limited basis,
- 5) observability – the degree to which the results of an innovation are visible to others.

Rogers postulates that innovations perceived by individuals as having less complexity but greater relative advantage, compatibility, trialability, and observability will be adopted more rapidly than other innovations. In Tornatzky and Klein’s meta-analysis consisting of 75 innovation characteristics studies, three characteristics (i.e., complexity, compatibility and relative advantage) were found to consistently influence innovation adoption and/or implementation. These three are included in our model.

RESEARCH MODEL AND HYPOTHESIS

Figure 1 shows our research model that is anchored in the aforementioned streams of research.

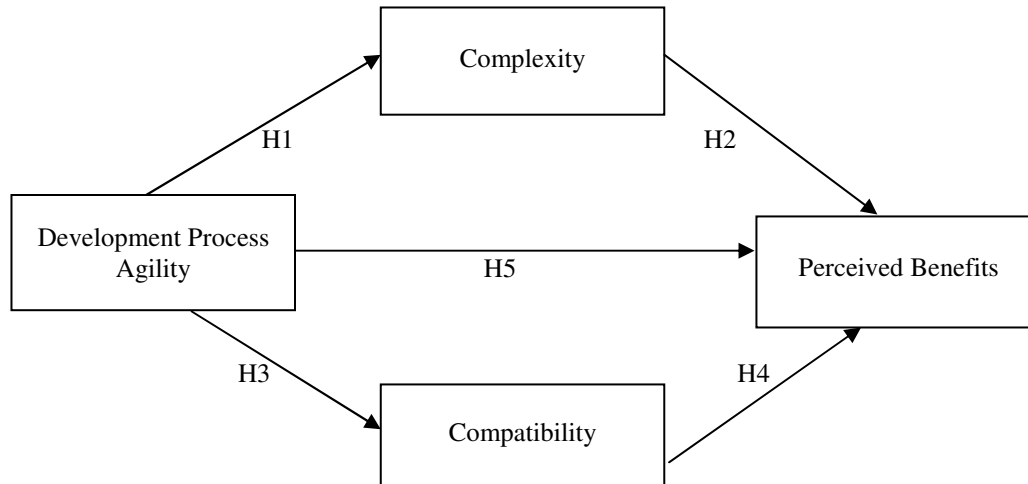


Figure 1. Research Model

When agile development methods are used, developers are cross-trained so that they are able to work on multiple facets of the project. This increases their efficacy regarding the ability to achieve desired outcomes related to the goal of producing working software. And, agile development is characterized by evolutionary development which ensures that evolving requirements and changes are continually accommodated. This is accomplished through an iterative, incremental development process that involves active engagement of customers (Beck 2000; Cockburn 2001; Highsmith 2004; Martin 2002). Feedback and communication are key elements of an evolutionary development process (Ambler 2000; Boehm and Turner 2004; Highsmith 2002, 2004), thus guiding the efforts of the developers. Developers quickly get a sense of what works and what doesn't and are continually learning and adjusting their behaviors. This process, therefore, enhances their skills and heightens their sense of competence leading to self efficacy regarding their use of the methodology. When developers have self efficacy regarding use of the methodology, we suggest that this leads to a belief that the methodology is easy to learn and use. Therefore, our first hypothesis addresses this negative relationship between process agility and complexity.

Hypothesis 1 (H1). *Process Agility of the development methodology will be negatively related to perceived complexity of the development methodology.*

Complexity was defined as the developer's perception of how hard or easy a methodology is to follow or learn. The less complex a system is, "the greater should be the user's sense of efficacy and personal control regarding his or her ability to carry out the sequences of behavior needed" (Davis et al. 1989). By contrast, a more complex system should result in a lower sense of efficacy regarding its use. Self-efficacy beliefs refer to people's judgments of their capabilities to execute courses of action that may be required to achieve a specific type or level of performance (Bandura, 1978). Self-efficacy beliefs have been shown to affect behaviors of people in that if they have a low feeling of self-efficacy regarding the ability to achieve the desired outcome, they will not expend as much effort to perform the action and in fact, may not even try at all (Bandura, 1986). Therefore, if individuals have low self-efficacy regarding their ability to follow the methodology, they may not expend much effort in trying to follow the methodology. This decreased effort may result in failure to achieve desired outcomes leading to lower perceptions of the benefits derived from using the methodology. The following hypothesis posits this relationship between complexity and perceived benefits.

Hypothesis 2 (H2). *Perceived Complexity of the development methodology will be negatively related to perceived benefits of using the development methodology.*

Agile methods prescribe that developers be empowered to make decisions without having to go through a bureaucratic approval process (Highsmith, 2002). There is not a rigid hierarchy or chain of command as agile leaders act more like coaches facilitating the development process (Chin, 2004; Cockburn and Highsmith, 2001). Because developers are given the freedom to be flexible and deviate from established processes in order to deliver working software (Highsmith, 2002; 2004) they have the autonomy to determine how they choose to accomplish their tasks. Since compatibility is defined as the degree to which developers feel that the methodology fits in with their way of doing things, this autonomy allows them to structure

the task to fit in with their way of doing things leading to a higher perception that the methodology is compatible. The following hypothesis addresses this relationship.

Hypothesis 3 (H3). *Process Agility of the development methodology will be positively related to perceived compatibility of the development methodology.*

As defined earlier, compatibility is the degree to which developers feel that the methodology fits in with their way of doing things. Compatibility has been reconceptualized as having multiple dimensions and it was found that compatibility with prior experience had an impact on ease of use (Karahanna et al. 2006). That is, if individuals felt that the system was compatible with their prior work experience, they felt that it was easier to use. So if developers felt that the methodology was compatible with their past work experience, they would find it easier to use and have greater amounts of efficacy with regard to its use. The increased feeling of efficacy, as discussed above, could result in increased effort which may contribute to a high level of success in achieving desired outcomes. Achievement of desired outcomes leads to an increase in perceived benefits of using the methodology. This positive relationship is the basis of our next hypothesis.

Hypothesis 4 (H4). *Perceived Compatibility of the development methodology will be positively related to perceived benefits of using the development methodology.*

Recent surveys show that practitioners consistently perceive that agile methodologies provide many benefits. These benefits include better communication and coordination, improved quality, greater productivity, and higher morale (Van Oyen, Gel, and Hopp, 2001; Versionone, 2008). A study at Microsoft also showed that similar benefits were realized by the users of agile methodologies (Begel and Nagappan, 2007). A white paper by Salesforce.com, a proponent of Cloud Computing, also highlights the significant improvements that agile development brought to their organization (Salesforce.com, 2009). Therefore, we posit that agile development processes result in perceived benefits. The following hypothesis relates process agility to perceived benefits.

Hypothesis 5 (H5). *Process Agility of the development methodology will be positively related to perceived benefits of using the development methodology.*

MEASUREMENTS

Within the domain of software development, a methodology's agility has become a primary focus because of the rapidly changing development environment. Agile methodologies are designed to aid the developer in responding quickly and effectively to these changes. Process agility has been conceptualized as comprising two dimensions: Evolutionary Development and Process Flexibility. Evolutionary Development captures the dynamic aspect of an agile process that strives to deliver working software in short iterations by anticipating and embracing change, actively involving stakeholders and continually seeking users' feedback to address their evolving needs. It embodies the collaborative dimension of agile processes coupled with the adaptability and responsiveness to changing conditions. Process Flexibility encompasses the reaction capabilities inherent in agile processes. It is characterized by flexibility of team members' roles and the willingness to deviate from established processes in order to deliver working software. It is the ability and willingness to improvise and come up with new solutions on the fly.

All scales have been adapted from previous studies. The scale for measuring process agility was developed and validated by (Bonner, 2008). The scales for complexity and perceived benefits were taken directly from a previous study on software development methodology (Riemenschneider et al. 2002). Measures for the various dimensions of compatibility were developed and validated by (Karahanna et al. 2006).

RESEARCH METHODOLOGY AND RESULTS

Data Collection

The data was collected using an online survey accessed via an Internet link. Subjects were software developers of all levels currently working in a software development capacity and using a software development methodology. There was no specific criteria related to the type of methodology and, in fact, in some cases they were using a combination of methods.

Prior to conducting the full scale study, the online survey was piloted with a total of 20 respondents. Most of the respondents were developers from industry and closely resembled the population of interest. Several of the respondents were academics with prior industry experience. Based on their comments, minor changes were made to the questionnaire. A preliminary assessment indicated adequate discriminant and convergent validity among the items. Therefore, no items were dropped from the survey.

We contacted CIO's and other top executives from several organizations via email and telephone. They were asked if they would consider sponsoring the survey by asking their developers to participate. They were informed about the general objective of the survey and assured that all responses would be kept confidential. If they agreed, they were provided with a link to the online survey. The introductory page displayed the university logo and a paragraph assuring their privacy and stating that participation was voluntary.

Because of the nature of online surveys, it is difficult to determine exact response rates for each organization. However, it is estimated that the overall number of subjects in the potential pool was approximately 8,000 and 554 surveys were completed. Therefore, we estimate a response rate of 6.92%. Large amounts of data were missing from 75 of the surveys, making it necessary to exclude them from the analysis. After dropping the 75 responses, our final sample size is 479.

Demographic information is presented in Table 1. The sample does seem to represent a wide selection of different industries, and a variety of age groups. Although a majority of the respondents are male, this is the case for the overall population of programmers/analysts (Craig, et al. 2002; Joshi and Schmidt 2005). As expected, most hold bachelor's or master's degrees. Therefore, we have good reason to believe that the sample comes from many different segments of the population, and is not a biased sample from a particular segment of the population. As illustrated, a variety of methodologies is represented. In order to allow for respondents that may be using a combination of methodologies, subjects were allowed to select more than one type.

DEMOGRAPHIC PROFILE OF RESPONDENTS			
MEASURE	ITEM	FREQ	PERCENT
Methodology	Adaptive Software Development	12	2.5
	Cleanroom	3	0.6
	CMM Integration	197	41.1
	Crystal	3	0.6
	Dynamic Systems Development	6	1.3
	Feature Driven Development	7	1.5
	Joint Application Development	19	4.0
	Lean Development	16	3.3
	Personal Software Process	7	1.5
	Rapid Application Development	43	9.0
	Rational Unified Process	114	23.8
	Scrum	10	2.1
	Spiral	28	5.8
	CMM for Software	50	10.4
	Team Software Process	11	2.3
	Waterfall	193	40.3
	eXtreme Programming	20	4.2
	Other	60	12.5
Country	North America	33	5.9
	India	382	79.7
	Canada	11	2.3
	England	15	3.1
	Bulgaria	16	3.3
	China	17	3.5
	Costa Rica	3	0.6
	France	1	0.2
	Germany	1	0.2
Gender	Female	73	15.2
	Male	392	81.8
Age	18 – 21 years	1	0.2
	21 – 30 years	261	54.5
	31 - 40 years	184	38.4
	41 – 50 years	15	3.1
	51 – 60 years	7	1.5

Education	Associate Degree	4	0.8
	Bachelor Degree	281	58.7
	Master Degree	176	36.7
	Doctorate Degree	1	0.2
	High School Degree	3	0.6
	Other	4	0.8
Industry	Banking/Insurance/Financial Service	12	2.5
	Construction/Architecture/Engineering	2	0.4
	Consulting/Business Service	65	13.6
	Government, including Military	2	0.4
	Health Care	4	0.8
	Hotel/Entertainment/Service Industry	4	0.8
	IT/Telecommunications	361	75.4
	Manufacturing	15	3.1
Job Title	Business Analyst	14	2.9
	Programmer	24	5.0
	Programmer/Analyst	146	30.5
	Manager/Supervisor	81	16.9
	Project Team Leader	128	26.7
Organization Size	Ranged from 6 to 600,000 Mean = 23,583		
Department Size	Ranged from 2 to 50,000 Mean = 2,110		
Percentage of organization Using Methodology	Less than 10%	9	1.9
	10 – 20	9	1.9
	20 – 30	22	4.6
	30 – 40	21	4.4
	40 – 50	27	5.6
	50 – 60	43	9.0
	60 – 70	67	14.0
	70 – 80	81	16.9
	80 – 90	57	11.9
	More than 90	118	24.6
Amount of Tool Support	Ranged from 1 = Minimal to 7 = Full Mean = 4.65		
Years with Organization	Ranged from Less than 1 to 28 Mean = 2.88		
Years Development Experience	Ranged from Less than 1 to 35 Mean = 6.47		
Months Using Methodology	Ranged from Less than 1 to 120 Mean = 29.5 months Less than 1 - 12 months = 127 (26.5%) 12 to 24 months = 139 (29%) (over half (55%) less than 2 years) 24 – 36 months = 74 (15.4%) 36 – 48 months = 50 (10.4%) 48 – 60 months = 34 (7.1%) Over 60 months = 32 (6.7%)		

Project Duration (in months)	Ranged from 1 to 168 Mean = 18.99 months		
Number of Team Members	Ranged from 1 to 500 Mean = 34.6		

Table 1. Demographic Profile of Respondents**Data Analysis**

We chose the Structural Equation Modeling (SEM) technique to analyze the data. Partial Least Squares (PLS) was used as our analysis method because it handles both formative and reflective indicators, whereas other SEM techniques do not. In addition, PLS is prediction oriented and it does not assume multivariate normality unlike SEM techniques such as LISREL. PLS Graph Version 3, Build 1130 (Courtesy of Dr. Chin, University of Houston) was the software tool used to conduct the analysis.

For the reflective constructs (complexity, compatibility and perceived benefits), we evaluated the measurement model by assessing the indicator reliabilities, the convergent validity of the indicators for each construct and the discriminant validity (see Tables 2 and 3). Indicators loading with a value less than .70 were dropped from the analysis with the exception of compatibility with prior experience which had a value of .69. All composite reliability values are above .70 indicating good internal consistency. The square root of the AVE scores is shown by the bolded diagonal elements in the table. The correlations between the constructs are illustrated by the off-diagonal elements. Since the square root of the AVE is higher than any correlations involving the construct our data demonstrates discriminant validity. For process agility, a formative construct, we used the procedure outlined by (Diamantopoulos and Winklhofer 2001). This procedure entails checking for collinearity and excluding those indicators that are highly collinear as they will inflate the variance explained by the latent variable (Bollen and Lennox 1991). The VIF was less than 3, which is much less than the heuristic of 10.

	Composite Reliability	AVE	Loading	Std. Error	T-Statistic
Compatibility	0.861	0.676			
COMPATEXP1			0.69	0.046	14.93
COMPATWORK1			0.87	0.019	46.08
COMPATWORK2			0.88	0.025	34.61
Complexity	0.914	0.680			
COMPL1R			0.84	0.27	31.42
COMPL2R			0.84	0.22	37.33
COMPL4R			0.89	0.012	70.21
COMPL5R			0.75	0.040	18.72
COMPL6R			0.77	0.035	22.14
Perceived Benefits	0.937	0.713			
PERBEN1			0.89	0.013	64.13
PERBEN2			0.91	0.011	81.27
PERBEN3			0.86	0.020	42.51
PERBEN4			0.83	0.029	27.83
PERBEN5			0.71	0.048	14.96
PERBEN6			0.83	0.024	33.46

Table 2. Indicator Loadings and Composite Reliabilities

	COMPAT	COMPLEX	PERCBEN
COMPAT	0.82		
COMPLEX	-0.67	0.82	
PERCBEN	0.67	-0.66	0.84

Table 3. AVE and Construct Correlations

DISCUSSION

Results

All hypotheses were supported. Figure 2 illustrates the magnitude of the relationships.

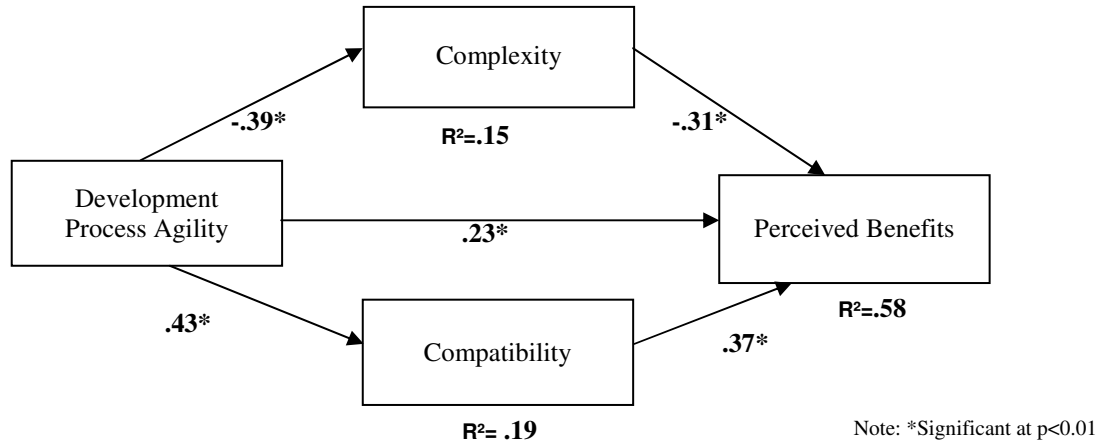


Figure 2. Structural Model

Implications for Research and Practice

This study has made valuable contributions in the area of software development and specifically in the area of systems development methodology implementation. Grounded in the agility and diffusion of innovations literature, this research introduces a measure of process agility within the software development domain and relates it to constructs previously employed in the innovation literature. For agile methods, our study has provided empirical support for the proposal that agile development methods lead to developers' beliefs that they are less complex, more compatible and provide increased benefits. Since developers believe that increased agility leads to increased benefits, they will be more likely to accept agile methods.

For practitioners, we have provided valuable insights into the underlying factors that influence a developer's beliefs about agile methods. This information will enable organizations to develop more effective strategies for choosing and implementing development methodologies. Our findings are welcome news to the practitioners of agile methodologies, since, indeed, they have positive and beneficial effects on developers' beliefs about their value which should lead to a higher level of acceptance.

Limitations

This study involved self-report surveys. In self-report studies, one limitation is the risk of common method bias, however some researchers doubt that the method alone causes systematic variance (Spector, 2006). Nonetheless, in order to assess the presence of common method bias, we conducted Harman's one-factor test (Podsakoff et al. 1984). Based on this analysis, there was no single factor which explained a substantial amount of the variance. These results present evidence for lack of common method bias, however the single factor test has been criticized by some as lacking sensitivity (Malhotra et al., 2006; Podsakoff et al., 2003). Consequently, we conducted the unmeasured latent method factor procedure (Podsakoff et al., 2003; Williams et al., 2003) outlined by Liang et al., 2007. It was found that the average variance of the indicators attributable to the method is .0126 while the average substantively explained variance is .50. These results illustrate that the method variance is low compared to the substantive variance, thus providing additional evidence that common method bias does not pose a significant threat to the measurement validity of this study.

In terms of generalizability, one possible limitation is that approximately 80% of our respondents were from one country with the remaining 20% being spread among various countries. However, no significant differences have been found in cross cultural studies related to the needs and motivations attributed to information technology workers (Couger and Motiwalla 1985; Couger et al. 1990). Additionally, we added culture as a control variable to the model and it did not have a significant effect.

REFERENCES

1. AgileAlliance "Manifesto for Agile Software Development," 2001 from <http://www.agilemanifesto.org> (accessed October 20, 2009).
2. "Agile Development Meets Cloud Computing for Extraordinary Results at Salesforce.com," a whitepaper from http://www.developerforce.com/media/ForcedotcomBookLibrary/WP_Agile_112608.pdf (accessed October 16, 2009)
3. Ambler, Scott. "Survey Says...Agile Has Crossed the Chasm," *Dr. Dobb's Journal*, July 2, 2007.
4. Ambler, Scott W. "The Extreme Programming Software Process Explained," *Computing Canada*, Vol. 26, Issue 5 (March 2000): 24-26.
5. Bandura, A. (1978), "Reflections on self-efficacy", *Advances in Behavioural Research and Therapy*, 1, 237-269.
6. Bandura, A. (1986). *Social foundations of thought and action: A social cognitive theory*. Englewood Cliffs, NJ: Prentice Hall.
7. Beck, K. (2000), *Extreme Programming Explained*, Addison-Wesley, New York.
8. Begel, A. and Nagappan, N., "Usage and Perceptions of Agile Software Development in an Industrial Context: An Exploratory Study," *Empirical Software Engineering and Measurement*, 2007. ESEM 2007. First International Symposium on 20-21 Sept. 2007 Page(s):255 - 264
9. Benko, C. and McFarlan, W. *Connecting the Dots: Aligning Projects with Objectives in Unpredictable Times*. Harvard Business School Press, Boston, 2003.
10. Boehm, B. and Turner, R. (2004), *Balancing Agility and Discipline*, Addison-Wesley, New York.
11. Bollen, K., and Lennox, R. (1991). "Conventional Wisdom on Measurement: A Structural Equation Perspective", *Psychological Bulletin*, 110(2), 305-314.
12. Bonner, Nancy A. "Developing an Instrument to Measure Process Agility in Software Development." Ph.D. diss., The University of Texas at Arlington, 2008.
13. Breu, Karin, Hemingway, Christopher J. and Strathern, Mark. (2001) "Workforce Agility: The New Employee Strategy for the Knowledge Economy", *Journal Of Information Technology*, 17, 21-31.
14. Cockburn, A. and J. Highsmith. (2001) "Agile Software Development: The People Factor", http://sunset.usc.edu/classes/cs510_2002/notes/agile_people_factor.pdf (accessed January 10, 2010).
15. Couger, J. D., Adelsberger, H., Borovits, I., Zviran, M. and Motiwalla, J. "Commonalities in Motivating Environments for Programmers/Analysts in Austria, Israel, Singapore, and the U.S.A.", *Information and Management* 18: 41-46 (1990).
16. Couger, J. Daniel, and Motiwalla, Juzar, "Occidental Versus Oriental I.S. Professionals' Perceptions on Key Factors for Motivation", *Proceedings of the Sixth International Conference on Information Systems*, Indianapolis, IN, December, 1985, pp. 105-112.

17. Craig, Annemieke, Paradis, Rose and Turner, Eva, "A Gendered View of Computer Professionals: Preliminary Results of a Survey", *ACM SIGCSE Bulletin*, Vol. 34, Issue 2 (June, 2002).
18. Davis, F.D., Bagozzi, R. and Warshaw, P. "User Acceptance of Computer Technology: A Comparison of Two Theoretical Models", *Management Science*, (35) 8 (1989), 982-1003.
19. Diamantopoulos, A., and Winklhofer, H. M. (2001). "Index Construction with Formative Indicators: An Alternative to Scale Development", *Journal of Marketing Research*, 38(2), 269-277.
20. Glass, R.L., "A snapshot of systems development practice", *IEEE Software*. 16 3 (1999) 110-111.
21. Hardgrave. B.C. "When to prototype: Decision variables used in industry", *Information and Software Technology*. 37. 2 (1995). 113-118.
22. Hartwick. J. and Barki. H. "Explaining the role of user participation in information system use", *Management Science*, 40. 4 (1994). 440-465.
23. Highsmith, J. (2004), *Agile Project Management*, Addison-Wesley, New York.
24. Highsmith, J. (2002), *Agile Software Development Ecosystems*, Addison-Wesley, New York.
25. IBM, 2009. "The New Voice of the CIO: Insights from the Global Chief Information Officer Study".
26. Joshi, K.D. and Schmidt, Nancy L. "Is the Information Systems Profession Gendered?: Characterization of IS Professionals and IS careers", *ACM SIGMIS Database*, v. 37 n.4, Fall, 2005.
27. Karahanna, E., Agarwal, R., and Angst C. "Reconceptualizing Compatibility Beliefs in Technology Acceptance," *MIS Quarterly*, 30:4, December 2006, pp. 781-804.
28. Kessler, Robert R. and Laurie A. Williams. "If this is What It's Really Like, Maybe I Better Major in English: Integrating Realism into a Sophomore Software Engineering Course," *Foundations in Education Conference*, 1999. Downloadable from <http://www.cs.utah.edu/~lwilliam/Papers/fie99.PDF> (accessed December 15, 2009)
29. Liang, H. G., Saraf, N., Hu, Q., & Xue, Y. J. "Assimilation of enterprise systems: The effect of institutional pressures and the mediating role of top management," *MIS Quarterly* 31(1), 2007, pp. 59-87.
30. Malhotra, Naresh K., Sung S. Kim and Ashutosh Patil, "Common Method Variance in IS Research: A Comparison of Alternative Approaches and a Reanalysis of Past Research," *Management Science*, (52:12), 2006, pp. 1865-1883.
31. Martin, Robert C. (2002), *Agile Software Development: Principles, Patterns, and Practices*, Prentice-Hall, New Jersey.
32. McDonald, J. (2001), "Why Is Software Project Management Difficult? And What That Implies for Teaching Software Project Management", *Computer Science Education*, 11, 1, 55-71.
33. Nagel, R. and Dove, R. (1991) *21st Century Manufacturing, Enterprise Strategy* (Iacocca Institute, Lehigh University, Bethlehem, PA).

34. Nosek, J.T. "The Case for Collaborative Programming," *Communications of the ACM*, Vol. 41, No. 3, 1998, pp. 105-108.
35. Parsons, D., Ryu, H. and Lal, R. 2007. "The Impact of Methods and Techniques on outcomes from Agile Software Development Projects", *IFIP 8.6 Conference: Organisational Dynamics of Technology-based Innovation: Diversifying the Research Agenda*, McMaster, Wastell, Ferneley and DeGross (eds.) Manchester UK, June 14-16 2007, Springer, 235-249.
36. Podsakoff, P. M., MacKenzie, S. B., Lee, J. Y., and Podsakoff, N. P. (2003). "Common Method Biases in Behavioral Research: A Critical Review of the Literature and Recommended Remedies". *Journal of Applied Psychology*, 88(5), 879-903.
37. Podsakoff, P. M., Todor, W.D., Grover, R.A., & Huber, V.L. (1984). "Situational moderators of leader reward and punishment behaviors: Fact or fiction?", *Organizational Behavior and Human Performance*, 34, 21-63.
38. Riemenschneider, C., Hardgrave, B.C. and Davis, D.D. (2002) "Explaining Software Developer Acceptance of Methodologies: A Comparison of Five Theoretical Models", *IEEE Transactions on Software Engineering*, 28, 12, pp. 1135-1145.
39. Swafford, Patricia M. Ph.D. Thesis, Georgia Institute of Technology, April, 2003.
40. Spector, P. E. (2006). "Method Variance in Organizational Research," *Organizational Research Methods* (9:2), pp. 221-232.
41. Tornatzky, L.G. and Klein, K.J., "Innovation Characteristics and Innovation Adoption-Implementation: A Meta-Analysis of Findings," *IEEE Transactions on Engineering Management*, Vol. EM-29, pp. 28-45, February, 1982.
42. Upchurch, Richard and Laurie Williams. "Extreme Programming for Software Engineering Education?," 31st ASEE/IEEE Frontiers in Education Conference 2001. "Downloadable from <http://collaboration.csc.ncsu.edu/laurie/>. (accessed December 16, 2009)
43. Van Oyen, Mark P., Gel, Esma G.S., and Hopp, Wallace J. (2001), "Performance Opportunity for Workforce Agility in Collaborative and Noncollaborative Work Systems", *IIE Transactions*, 33, 761-777.
44. VersionOne, (2008). "3rd Annual Survey: 2008 'The State of Agile Development'", June-July, 2008.
45. Williams, Laurie Ann. "The Collaborative Software Process." Ph.D. diss., The University of Utah, 2000.
46. Williams, L. J., Edwards, J. R. and Vandenberg, R. J. "Recent advances in causal modeling methods for organizational and management research," *Journal of Management* (29:6), 2003, pp. 903-936.
47. Williams, Laurie, Kessler, Robert R., Cunningham, Ward and Jeffries, Ron. (2000), "Strengthening the Case for Pair Programming", *IEEE Software*, July/August, 2000, 19-25. Rogers, Everett (2003, 1995, 1983, 1971, 1962), *Diffusion of Innovations, Fifth Edition*, Free Press, New York.